EE 446 TinyML
Homework 2 Writeup

Kourosh LastName

August 12, 2026

Setup

The Network Anomaly Dataset was loaded as 125,973 samples with 43 columns. After dropping `land`, `urgent`, `numfailedlogins`, and `numoutboundcmds`, collapsing all non-`normal` labels to `attack`, and label-encoding the four categorical columns, 39 columns remained: 38 features and the binary target. `LabelEncoder` assigned `attack` $\rightarrow 0$ and `normal` $\rightarrow 1$. Features were standardized and split 80/20, giving `X_train` of shape $(100778, 38)$ and `X_test` of shape $(25195, 38)$. A fixed `random_state` was added to `train_test_split` so that the test samples deployed to the Arduino remain reproducible across notebook runs.

Question 2: Dimensionality Reduction for Visualization

All three projections mark attack points in red and normal points in blue. t-SNE and Kernel PCA were computed on a random subsample of 2,000 test points, since t-SNE scales poorly and the rbf Kernel PCA requires an $n \times n$ kernel matrix that is not tractable for the full test set.

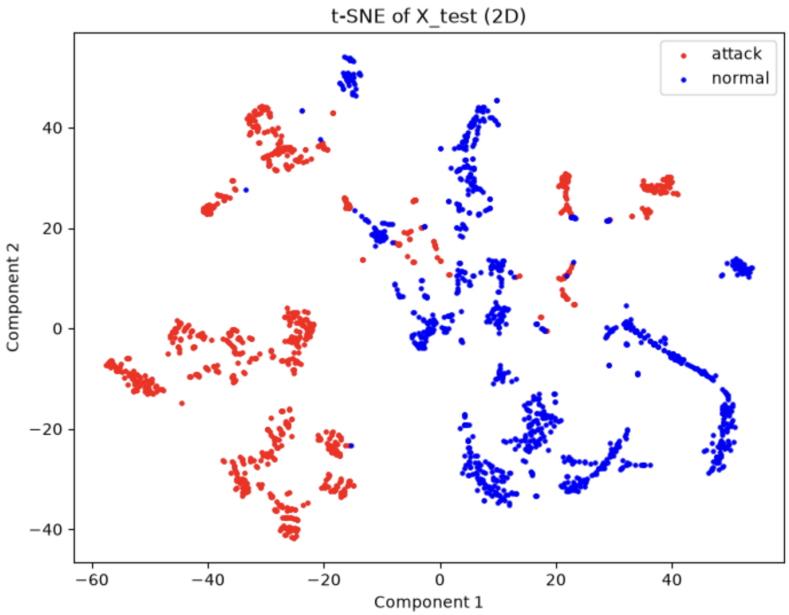

Figure 1: Question 2(a): t-SNE projection of the test set in 2D.

t-SNE separates the two classes almost completely, with attack points occupying the left of the embedding and normal points the right, and only isolated points crossing between them. The many small islands within each class reflect the distinct attack types that were merged into a single label during preprocessing.

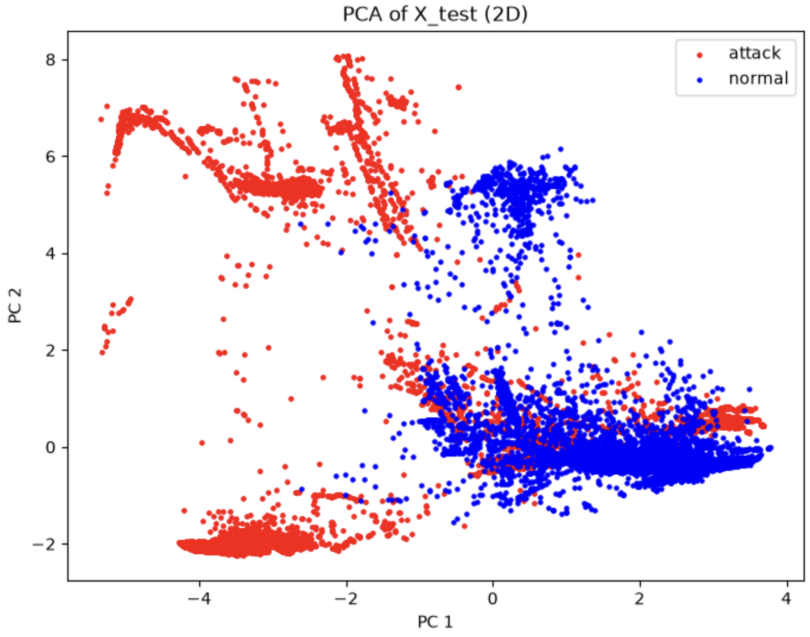

Figure 2: Question 2(b): PCA projection of the test set in 2D.

PCA separates the classes only partially. A large attack mass sits at negative PC1 and a normal mass at positive PC1, but the two overlap heavily in the lower-right region. This is expected, since PCA is a linear projection that maximizes variance rather than class separation.

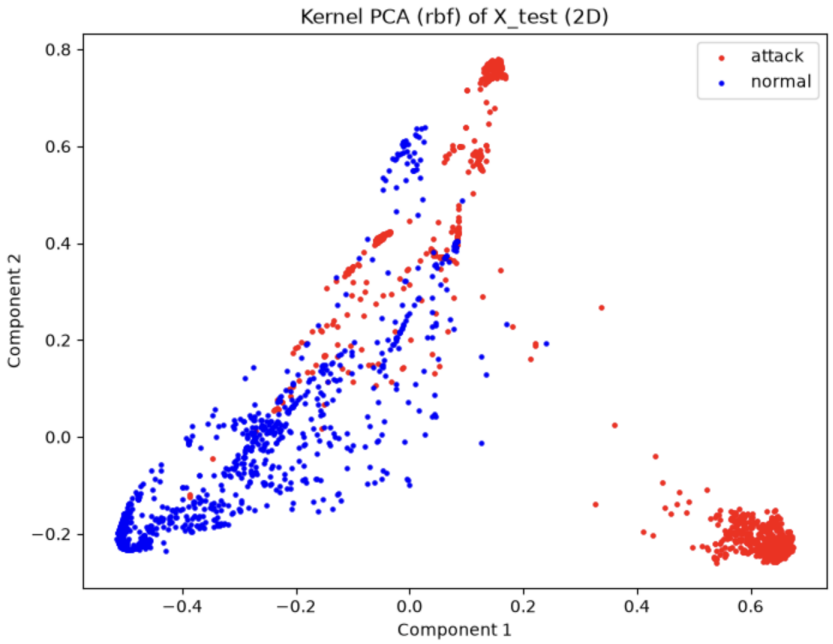

Figure 3: Question 2(c): Kernel PCA (rbf) projection of the test set in 2D.

Kernel PCA falls between the two. A tight attack cluster and a well-isolated normal arm are recovered, but a diagonal band through the middle of the embedding remains mixed. The rbf kernel captures nonlinear structure that PCA misses without reaching the neighbor-level resolution of t-SNE.

Question 3: Baseline DNN

The network is a fully connected model with two hidden layers of 32 and 16 ReLU units and a single sigmoid output neuron, totalling 1,793 parameters. It was compiled with the Adam optimizer at a learning rate of 0.001 and binary cross-entropy loss, then trained for 10 epochs with a batch size of 64 and a 10% validation split. Training accuracy reached 0.9983 with a validation accuracy of 0.9986.

```
788/788 [==============================] - 0s 230us/step
              precision    recall  f1-score   support

      attack     0.9980    0.9984    0.9982     11736
      normal     0.9986    0.9982    0.9984     13459

    accuracy                         0.9983     25195
   macro avg     0.9983    0.9983    0.9983     25195
weighted avg     0.9983    0.9983    0.9983     25195

[[11717    19]
 [   24 13435]]
```

Figure 4: Question 3(c): classification report and confusion matrix for `base_model` on the test set.

The float32 model achieves 0.9983 accuracy on the 25,195 test samples, misclassifying 43 samples

in total: 19 attacks predicted as normal and 24 normal connections predicted as attacks. The near-complete class separation seen in the t-SNE plot is consistent with this result.

Question 4: Full-Integer Quantized Model

Post-training quantization was performed with `TFLiteConverter` using `Optimize.DEFAULT`, a representative dataset of 500 training samples to calibrate activation ranges, the `TFLITE_BUILTINS_INT8` operator set, and INT8 inference input and output types. The converter reported the input tensor scale as 0.09267 with zero point -36, and the output tensor scale as 0.00390625 with zero point -128; the latter maps the sigmoid output range exactly onto the INT8 range.

```
Original model size: 51.80 KB
Quantized model size: 4.30 KB
```

Figure 5: Question 4(a): original and quantized model file sizes.

```
input scale/zp: 0.09267217665910721 -36
output scale/zp: 0.00390625 -128
              precision    recall  f1-score   support

      attack     0.9977    0.9986    0.9982     11736
      normal     0.9988    0.9980    0.9984     13459

    accuracy                         0.9983     25195
   macro avg     0.9983    0.9983    0.9983     25195
weighted avg     0.9983    0.9983    0.9983     25195

[[11720    16]
 [   27 13432]]
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

Figure 6: Question 4(b): classification report and confusion matrix for `tflite_quant_model` on the test set, with the reported input and output quantization parameters.

Each test sample was quantized with the input tensor's scale and zero point before inference, and the INT8 output was dequantized back to a probability before thresholding at 0.5. The quantized model matches the float32 baseline at 0.9983 accuracy, again with 43 total errors, redistributed to 16 attacks predicted as normal and 27 normal connections predicted as attacks. Full-integer quantization therefore costs no measurable accuracy on this dataset while reducing the model from 51.80 KB to 4.30 KB, a factor of roughly twelve. The resulting 4,408-byte flatbuffer is what makes deployment to the Arduino Nano 33 BLE Sense feasible.

Question 5: Deployment on the Arduino Nano 33 BLE Sense

The prediction is recovered from the output tensor by dequantizing the single INT8 value with the tensor's own scale and zero point, which yields the sigmoid probability directly; the predicted

class is 1 when that probability exceeds 0.5 and 0 otherwise. Reading the quantization parameters from the tensor at runtime rather than hard-coding them keeps the sketch correct if the model is reconverted. Sample number, predicted class, and actual class are printed to the serial monitor at 115200 baud for each sample.

```
16:47:42.058 -> Sample #0 | Predicted Class: 1 | Actual Class: 1
16:47:43.079 -> Sample #1 | Predicted Class: 0 | Actual Class: 0
16:47:44.066 -> Sample #2 | Predicted Class: 0 | Actual Class: 0
16:47:45.089 -> Sample #3 | Predicted Class: 1 | Actual Class: 1
16:47:46.080 -> Sample #4 | Predicted Class: 1 | Actual Class: 1
```

Figure 7: Question 5(c): serial monitor output for the first five test samples.

All five predictions match their actual labels.

```
17:02:46.403 -> Sample #0 | Predicted Class: 0 | Actual Class: 0
17:02:47.394 -> Sample #1 | Predicted Class: 1 | Actual Class: 1
17:02:48.382 -> Sample #2 | Predicted Class: 1 | Actual Class: 1
17:02:49.404 -> Sample #3 | Predicted Class: 0 | Actual Class: 0
17:02:50.392 -> Sample #4 | Predicted Class: 1 | Actual Class: 1
17:02:51.411 -> Sample #5 | Predicted Class: 1 | Actual Class: 1
17:02:52.402 -> Sample #6 | Predicted Class: 1 | Actual Class: 1
17:02:53.392 -> Sample #7 | Predicted Class: 0 | Actual Class: 0
17:02:54.414 -> Sample #8 | Predicted Class: 0 | Actual Class: 0
17:02:55.403 -> Sample #9 | Predicted Class: 0 | Actual Class: 0
```

Figure 8: Question 5(d): serial monitor output for the ten test samples following the first five.

The ten samples drawn from `X_test[5:15]` are also classified correctly, giving 15 of 15 correct on-device predictions. This is consistent with the 0.9983 test accuracy measured for the quantized model in the notebook and confirms that the deployed C array, the runtime input quantization, and the output dequantization on the board reproduce the behavior observed in Python.